数学与 CS 问答笔记

math-answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

October 04, 2026

Contents

1. 当日起点：项目与工具状态 5
   1.1. db-go：KV 第 1 步正在进行（你自己在写） 5
      1.1.1. 写 `Get` 时的一个易错点 5
   1.2. 工具：四个 godoc 函数已就位 5
   1.3. 待办 6
2. `map[string][]byte{}` 背后调用了哪些 API 7
   2.1. 费曼直觉 7
   2.2. 硬核机制 7
      2.2.1. 查询方法：把语法编译成汇编 7
      2.2.2. 映射表：语法与运行时函数 8
      2.2.3. 为什么空字面量没有 makemap 8
      2.2.4. 顺带发现：Go 的 map 已经换成 Swiss table 9
   2.3. 陷阱与易错 9
   2.4. 关键点 10
   2.5. 变式检验 10
   2.6. 同类归类 10
   2.7. Go 语言机制补充 10
      2.7.1. 逃逸分析与“栈上分配”的关系 10
      2.7.2. 怎么读 `-S` 的输出 11
3. 该查哪个 API，以及怎么读懂它的返回值 12
   3.1. 费曼直觉 12
   3.2. 硬核机制 12
      3.2.1. 实测：runtime 那些函数你根本调不了 12
      3.2.2. 语言结构 ↔ 配套标准库包 13
      3.2.3. 三个查询场景 13
      3.2.4. 怎么读懂“返回值”：四个信息源 13
   3.3. 陷阱与易错 14
   3.4. 关键点 14
   3.5. 变式检验 14
   3.6. 同类归类 15
   3.7. Go 语言机制补充 15
      3.7.1. 为什么这些函数的签名那么长 15
      3.7.2. 判断一个标识符“能不能用”的两条规则 15
4. 怎么用 go doc 查 map 相关的内容 17
   4.1. 费曼直觉 17
   4.2. 硬核机制 17
      4.2.1. 三个反直觉的实测结果 17
      4.2.2. 路径一：内建操作 —— go doc builtin 18
      4.2.3. 路径二：实用函数 —— go doc maps 18
      4.2.4. 路径三：并发安全 map —— go doc sync.Map 19
      4.2.5. 路径四：语言语义 —— 只能看规范 19

4.3. 陷阱与易错 20
4.4. 关键点 20
4.5. 变式检验 20
4.6. 同类归类 20
4.7. Go 语言机制补充 20
4.7.1. builtin 包是个“文档用的假包” 20
4.7.2. 预声明标识符与关键字的区别 21
5. 逐段读懂一个泛型函数声明 22
5.1. 费曼直觉 22
5.2. 硬核机制 22
5.2.1. 第一段：func All 22
5.2.2. 第二段：[Map map[K]V, K comparable, V any] 22
5.2.3. 第三段：(m Map) 23
5.2.4. 第四段：iter.Seq2[K, V] 23
5.2.5. 怎么用这个返回值（两种写法，都实测过） 23
5.2.6. 从简到繁：函数声明的六个台阶 24
5.3. 陷阱与易错 24
5.4. 关键点 24
5.5. 变式检验 25
5.6. 同类归类 25
5.7. Go 语言机制补充 25
5.7.1. 约束本质上是一个接口 25
5.7.2. 类型参数列表的语法 25
6. 类型参数（泛型）到底是做什么用的 27
6.1. 费曼直觉 27
6.2. 硬核机制 28
6.2.1. 三条路：写三遍、用 any 硬塞、用泛型 28
6.2.2. 实测：B 和 C 的差别在哪 28
6.2.3. 于是“类型参数”是什么，可以一句话回答 29
6.2.4. 但别急着到处用：标准库自己就很克制 30
6.2.5. 接口与泛型的分工 30
6.3. 陷阱与易错 30
6.4. 关键点 30
6.5. 变式检验 31
6.6. 同类归类 31
6.7. Go 语言机制补充 31
6.7.1. 泛型的实例化方式：按“形状”共享代码 31
6.7.2. 约束决定了函数体内能做什么 31
7. `kv.mem[string(key)]` 到底返回什么 32
7.1. 费曼直觉 32
7.2. 硬核机制 32
7.2.1. map 索引的返回值规则（规范原文） 32

7.2.2. 于是有且只有两种写法 33
7.2.3. 顺带盘点：Go 里只有这三种表达式能“给两个值” 33
7.2.4. 那“怎么知道任意表达式的返回值”？三条路 34
7.2.5. 正确写法 34
7.3. 陷阱与易错 35
7.4. 变式检验 35
7.5. 同类归类 35
7.6. Go 语言机制补充 35
7.6.1. 为什么 map 索引不返回 error 35
7.6.2. comma-ok 只能在“赋值/初始化”位置生效 36
8. 怎么查 `:=` 与 `=` 的区别 37
8.1. 费曼直觉 37
8.2. 硬核机制 37
8.2.1. 规范地图：19 个大章 37
8.2.2. 规范的原文（两节对照） 37
8.2.3. 一句话区分 38
8.2.4. 三条硬规则（都实测过） 38
8.2.5. 回到你的 `Get`： 为什么必须用 `=` 39
8.3. 陷阱与易错 39
8.4. 关键点 40
8.5. 变式检验 40
8.6. 同类归类 40
8.7. Go 语言机制补充 40
8.7.1. 怎么在规范里快速定位 40
8.7.2. 规范里语法产生式的命名规律 41
Index of Listings 42

1. 当日起点：项目与工具状态

这一节记录 10-04 开工时的状态，当作今天的基线；后续问答按约定追加在它下面。

1.1. db-go：KV 第 1 步正在进行（你自己在写）

你已把 `internal/db/kv.go` 回到"练习模式"并动手实现。当前编译只有一处阻塞：

```
$ go build ./...
# github.com/zz-cpp/db-go/internal/db
internal/db/kv.go:15:1: missing return
```

Listing 1: 当前唯一的编译错误：Get 的函数体为空

四个方法的现状：

方法	状态	说明
`Open`	已完成	已重新写上 `kv.mem = map[string][]byte{}` ——这行不能省，nil map 写入会 panic（10-01 那节有实测）
`Get`	缺函数体	`missing return`：声明了具名返回值就必须有 `return` 语句，哪怕是返回零值
`Set`	待完善	目前是 `return true, nil`；与课程语义不同—— `updated` 表示"值真的变了"，应写成 `!exist || !bytes.Equal(prev, val)`
`Del`	还没写	只有签名，编译到它时会报 `missing function body`

> 📌 **核心要点** 两条提示，都能在 10-01 的《补全 KV 的三个方法》那节找到完整依据：① 具名返回值的函数必须有 `return`；② `Set` 的 `updated` 不是"写入成功"，而是"值发生了变化"——重复写入相同的值应当返回 `false`。

1.1.1. 写 `Get` 时的一个易错点

`Get` 的返回值是 `(val []byte, ok bool, err error)`，三个都要给。用命名返回值可以让它很短：

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
  val, ok = kv.mem[string(key)]
  return
}
```

Listing 2: 命名返回值 + 裸 return：val/ok 已被赋值，err 自动为零值 nil

注意 `val, ok = ...` 用的是 `=` 而不是 `:=`——`val` 和 `ok` 已经在签名里声明过了，`:=` 会变成"重新声明"从而编译报错 `no new variables on left side of :=`。

1.2. 工具：四个 godoc 函数已就位

函数	等价命令	用途
`godocbat`	`go doc <参数>`	包概览 / API / 类型 / 方法（最常用）

`godocall`	`go doc -all <包>`	整个包的完整文档
`godocsrc`	`go doc -src <符号>`	源码实现
`godochelp`	（自述）	显示函数家族的用法

四个文件都在 `~/dotfiles/fish/.config/fish/functions/`，fish 的 autoload 让它们立即可用（细节见 10-03 那份笔记）。

1.3. 待办

1. 补 `Get` 的函数体——当前编译阻塞点；
2. 补 `Del` 的函数体（提示：`delete` 对不存在的键是安全空操作，所以要先查一次拿到 `deleted`）；
3. 视需要完善 `Set` 的 `updated` 语义（想严格对齐课程就改，想先跑通测试也可以后补）；
4. 全部补完后按四步验证：`gofmt -l .` → `go build ./...` → `go vet ./...` → `go test ./...`。

2. `map[string][]byte{}` 背后调用了哪些 API

2.1. 费曼直觉

先纠正前提：**这一行在源码层面不调用任何 API。** `map[string][]byte{}` 是**复合字面量**（composite literal），不是函数调用——它没有括号参数、没有函数名。

真正"调用"发生在翻译阶段：**编译器**把这行语法翻译成一串对 `runtime` 包的调用。所以要回答"调用了哪些 API"，要看的不是源码，而是**编译产物**。

2.2. 硬核机制

2.2.1. 查询方法：把语法编译成汇编

两条互补的路子：

想知道什么	用什么
某段**语法**背后是什么调用	`go build -gcflags=-S`（看汇编，输出带源码行号）
某个**运行时函数**是做什么的	`go doc -u runtime.<名字>`（`-u` 才能看见未导出函数），再到 `/usr/lib/go/src/runtime/` 读源码

先做个最小程序，把六种 map 操作放进去：

```
package main

func main() {
  m := map[string][]byte{}          // ① 复合字面量（空 map）
  v, ok := m["k"]                   // ② 读取
  _, _ = v, ok
  m["k"] = []byte("v")              // ③ 写入
  delete(m, "k")                    // ④ 删除
  for k := range m {                // ⑤ 遍历
    _ = k
  }
  n := make(map[string][]byte, 100) // ⑥ make 带容量
  _ = n
}
```

Listing 3: 六种 map 操作的探针程序

然后一条命令提取全部调用：

```
    $ go build -gcflags=-S ./... 2>&1 | grep -oE 'CALL runtime\.[a-zA-Z0-9_]+' | sort | uniq
-c | sort -rn
          1 CALL runtime.rand
          1 CALL runtime.mapIterStart
          1 CALL runtime.mapIterNext
          1 CALL runtime.mapdelete_faststr
          1 CALL runtime.mapassign_faststr
          1 CALL runtime.mapaccess2_faststr
          1 CALL runtime.mallocgcTinySC2
          1 CALL runtime.makemap
          1 CALL runtime.gcWriteBarrier2
```

Listing 4: 六种语法 → runtime 调用，一一对应

2.2.2. 映射表：语法与运行时函数

源码写法	编译后的 RUNTIME 调用
`map[K]V{}`（空字面量）	**不调用 makemap**——见下一小节，编译器把它放在栈上；只调 `runtime.rand` 取哈希种子
`make(map[K]V, n)`	`runtime.makemap`（带容量提示）
`m[k]`（单值形式）	`runtime.mapaccess1`（string 键会特化成 `mapaccess1_faststr`）
`v, ok := m[k]`（双值形式）	`runtime.mapaccess2_faststr`
`m[k] = v`	`runtime.mapassign_faststr`（字符串值还要 `gcWriteBarrier2`）
`delete(m, k)`	`runtime.mapdelete_faststr`
`for k := range m`	`runtime.mapIterStart` + `mapIterNext`（迭代器）

三个值得留意的细节：

1. **`_faststr` 后缀**是编译器为 `map[string]X` 生成的**特化版本**——专门优化字符串键的哈希与比较。整数键对应 `_fast64` / `_fast32`，其他类型则走通用版 `mapassign` 等；
2. **`runtime.rand`** 是哈希**种子**：Go 给每个 map 一个随机种子，让攻击者无法预测哈希分布（防哈希碰撞拒绝服务）；
3. **`gcWriteBarrier2`** 是写屏障——GC 的记账机制，往 map 里塞指针类型（这里是 `[]byte`）时必须登记。

2.2.3. 为什么空字面量没有 makemap

看第 4 行（`m := map[string][]byte{}`）生成的汇编：

```
0x001d 00029 (/tmp/mapapi/main.go:4)  LEAQ  main..autotmp_11+416(SP), AX
0x0025 00037 (/tmp/mapapi/main.go:4)  MOVUPS  X15, (AX)
...（一串向栈内存清零的 MOVUPS）...
0x005d 00093 (/tmp/mapapi/main.go:4)  MOVQ  $-9187201950435737472, AX
0x006c 00108 (/tmp/mapapi/main.go:4)  LEAQ  main..autotmp_12+88(SP), AX
0x0071 00113 (/tmp/mapapi/main.go:4)  MOVQ  AX, main..autotmp_11+432(SP)
```

Listing 5: 整个字面量都在操作 SP（栈指针）——这是栈上分配的 map，没有堆分配、没有 makemap 调用

`SP` 是栈指针，`main..autotmp_11` / `autotmp_12` 是编译器自动命名的栈临时变量。整行代码都在栈上完成，因为**编译器能证明这个 map 不会逃逸出函数**。`go doc -u runtime.makemap` 的文档也证实了这一点：

```
makemap implements Go map creation for make(map[k]v, hint). If the compiler
has determined that the map or the first group can be created on the stack,
m and optionally m.dirPtr may be non-nil.
```

> 📌 **核心要点** 同一个 `=` 赋值，写法不同、运行时成本可能完全不同：`map[string][]byte{}` 可以被编译器放到**栈上**（零堆分配），而 `make(map[string][]byte, 100)` 会调 `runtime.makemap` 分配哈希表。这就是 `Open()` 里那行初始化“几乎免费”的原因。

2.2.4. 顺带发现：Go 的 map 已经换成 Swiss table

查 `go doc -u runtime.makemap` 时，签名里的 `*maps.Map` 是个线索——它指向 `/usr/lib/go/src/internal/runtime/maps/`。那个包的注释写着：

```
// This package contains the implementation of Go's builtin map type.
//
// The map design is based on Abseil's "Swiss Table" map design
// (https://abseil.io/about/design/swisstables), with additional modifications
// to cover Go's additional requirements, discussed below.
//
// Terminology:
// - Slot: A storage location of a single key/element pair.
// - Group: A group of abi.MapGroupSlots (8) slots, plus a control word.
// - Control word: An 8-byte word which denotes whether each slot is empty,
//   deleted, or used. If a slot is used, its control byte also contains the
//   lower 7 bits of the hash (H2).
// - H1: Upper 57 bits of a hash.
// - H2: Lower 7 bits of a hash.
```

而汇编里那句 `MOVQ $-9187201950435737472, AX`（即 `0x8080808080808080`）就是 Swiss table 的**控制字**初值——每个字节的最高位为 1 表示“空槽”。所以本机 go1.27.1 用的已经是这套新实现（2024 年引入），不再是老教材里的 `hmap` + `bmap`。

2.3. 陷阱与易错

1. **在源码里找“函数调用”。**复合字面量、`make`、`delete`、`range` 都是**语言结构**，它们的“API”只存在于编译产物里；

2. **`go doc runtime.makemap` 查不到**。这些是未导出函数，必须加 `-u`：`go doc -u runtime.makemap`；
3. **以为 `runtime` 的函数在 `map.go` 里**。`map_faststr.go` 里的函数都是 `//go:linkname` 的存根，真正的实现在 `internal/runtime/maps/`；
4. **拿老资料对照新版本**。Go 1.24 起 map 实现换成了 Swiss table，网上大量"hmap / bmap / 桶溢出"的讲解已经不适用于当前版本——**这正是优先用本机 `go doc` 而不是搜索引擎的原因**。

2.4. 关键点

> 📌 **核心要点** 两类问题、两套工具：问"这段**语法**变成了什么" → `go build -gcflags=-S`（汇编带行号）；问"这个**运行时函数**做什么" → `go doc -u runtime.X` + 读 `/usr/lib/go/src/runtime/`。

2.5. 变式检验

1. **换成其他模板类型**：把值类型从 `[]byte` 换成不含指针的 `int`，`gcWriteBarrier2` 就会消失（GC 不需要登记不含指针的值）——这是观察"写屏障何时出现"的好办法；
2. **换成整数键**：`map[int]int{}` 的调用会变成 `mapaccess2_fast64` / `mapassign_fast64`，验证了 `_faststr` 只是特化系列中的一种；
3. **换成把 map 返回出去**：写 `func New() map[string][]byte { return map[string][]byte{} }` 再看汇编，会发现 `makemap` 回来了——因为 map 逃逸出函数，不能再放栈上。

2.6. 同类归类

1. 与 C 的对照：C 里 `int a[10] = {0};` 是纯栈操作，没有运行时调用；Go 的 `map` 因为要处理哈希、扩容、GC，才需要 runtime 配合；
2. 与 Python 的对照：`{}` 在 CPython 里会调 `BUILD_MAP` 字节码指令 + `PyDict_New` ——同样存在"语法 → 虚拟机/运行时调用"的映射，只是 Python 的映射在字节码层面可见（`dis` 模块），Go 的要到汇编层面才看到；
3. 往下延伸 本课程后面自己实现存储引擎时，也会做类似的分层——上层语法（如 `Set`）与底层机制（日志写入、fsync、页缓存）分离。理解"编译器替我们做了哪些翻译"，是理解这种分层的前提。

2.7. Go 语言机制补充

2.7.1. 逃逸分析与"栈上分配"的关系

默认情况下 `make` 出来的 map 在堆上，因为 map 是引用类型、通常要活得比函数长。但编译器会做**逃逸分析**：如果一个 map 从创建到死亡都待在同一函数里、地址没被传出去，它就可以被放到栈上——分配/回收随栈帧自动完成，不惊动 GC。

本例中的空字面量正是这种情况：`m` 只在 `main` 里用，没有传出去，于是整块结构都在 `SP` 相对寻址的栈空间里。这也解释了为什么汇编里没有 `mallocgc`（堆分配）——只有 `[]byte("v")` 那个小对象因为要存进 map 而调了 `mallocgcTinySC2`。

2.7.2. 怎么读 `-s` 的输出

`-gcflags=-S` 每行格式是：

```
0x0071 00113 (/tmp/mapapi/main.go:4)  MOVQ  AX, main..autotmp_11+432(SP)
  ↑偏移    ↑机器码长度    ↑源码位置          ↑指令      ↑操作数
```

`(main.go:4)` 这个标注是关键——它让“哪行源码产生了这条指令”一目了然。想只看某个函数时，用 `sed -n '/TEXT.*main\.main/,/RET/p'` 截取；想过滤调用，用 `grep 'CALL'`；想知道某个运行时函数被谁调用，直接 `grep 'CALL runtime\.makemap'`。

> 📌 **核心要点** 一句话收束：**Go 的“语法”与“运行时 API”之间隔着一层编译器翻译。**想看清这层翻译，`go build -gcflags=-S` 是最短路径；想看清被调用的函数本身，`go doc -u` 加 runtime 源码是标配。

3. 该查哪个 API，以及怎么读懂它的返回值

3.1. 费曼直觉

上一节查出了 `map` 背后的 `runtime.makemap`、`mapassign_faststr`——但**那些不是给你用的 API**。这节先破这个误区，再给出真正可用的查法。

一句话区分三类东西：

类别	例子	你能调用吗
语言语法	`m[k]`、`delete(m, k)`、`for range m`	这就是语法本身，没有“函数 API”可查
运行时内部函数	`runtime.makemap`、`mapassign_faststr`	**不能**——未导出，编译器与 runtime 的私有契约
标准库导出 API	`maps.Clone`、`slices.Sort`、`strings.Builder`	能——这才是写代码时要查的

3.2. 硬核机制

3.2.1. 实测：runtime 那些函数你根本调不了

```
package main

import "runtime"

func main() {
  runtime.makemap(nil, 0, nil)
  runtime.mapassign_faststr(nil, nil, "")
}
```

Listing 6: 试着直接调用上一节看到的 runtime 函数

```
$ go build ./...
# probe
./main.go:6:10: undefined: runtime.makemap
./main.go:7:10: undefined: runtime.mapassign_faststr
```

Listing 7: 报的是 undefined——原因正是前面讲过的导出规则：小写开头，外部看不见

就算你能看见，也调不了：它们的参数是 `*abi.MapType`、`unsafe.Pointer` 这种**编译器内部类型**，是编译器与 runtime 之间的约定，不是给人类用的接口。

> 📌 **核心要点** 上一节查 `makemap` 是为了理解**机制**（编译器怎么翻译语法）；这一节要查的是**能用的 API**。两件事的目标不同，查的对象也不同——前者指向 `runtime`，后者应该指向标准库的**配套包**。

3.2.2. 语言结构 ↔ 配套标准库包

Go 近年在给“内置类型”补齐可调用的 API，形成一组对应关系：

语言结构	配套包	导出符号数	典型函数
`map[K]V`	`maps`	10	`Clone` 、 `Copy` 、 `Equal` 、 `Keys` 、 `Values` 、 `DeleteFunc`
`[]T` 切片	`slices`	40	`Sort` 、 `Contains` 、 `Index` 、 `Delete` 、 `Compact` 、 `Clone`
`string`	`strings`	61	`Split` 、 `Join` 、 `TrimSpace` 、 `Builder` 、 `Replace`
比较	`cmp`	4	`Compare` 、 `Less` 、 `Or`
排序（老接口）	`sort`	22	`Slice` 、 `SliceStable` 、 `Search` 、 `Interface`

所以“我想对 map 做某件内置语法做不到的事”，答案几乎总在 `maps` 包里，而不是在 `runtime` 里。

3.2.3. 三个查询场景

```
    # ① 想看这个包能做什么（包概览，一行摘要）
    $ godocbat maps
    func All[Map ~map[K]V, K comparable, V any](m Map) iter.Seq2[K, V]
    func Clone[M ~map[K]V, K comparable, V any](m M) M
    func Collect[K comparable, V any](seq iter.Seq2[K, V]) map[K]V
    … (共 10 个)

    # ② 想找'能做某件事'的函数（全包 + 关键词）
    $ go doc -all maps | grep '^func ' | grep -iE 'copy|clone|equal'
    func Clone[M ~map[K]V, K comparable, V any](m M) M
    func Copy[M1 ~map[K]V, M2 ~map[K]V, K comparable, V any](dst M1, src M2)
    func Equal[M1, M2 ~map[K]V, K, V comparable](m1 M1, m2 M2) bool
    func EqualFunc[M1 ~map[K]V1, M2 ~map[K]V2, K comparable, V1, V2 any](m1 M1, m2 M2, eq
func(V1, V2) bool) bool

    # ③ 想判断某个具体函数怎么用、返回什么
    $ godocbat maps.Clone
```

Listing 8: 从“有什么”到“怎么用”的三步

3.2.4. 怎么读懂“返回值”：四个信息源

用户真正的诉求是“看 API 判断如何用它”，而返回值的语义主要藏在**文档注释**里，不只是签名。以 `maps.Clone` 为例：

```
    $ godocbat maps.Clone
    func Clone[M ~map[K]V, K comparable, V any](m M) M
        Clone returns a copy of m. This is a shallow clone: the new keys and values
        are set using ordinary assignment.
```

Listing 9: 签名说“返回 M”，注释说“是浅拷贝”——后者才是决定你怎么用的关键

四个信息源，按可靠度排：

信息源	能告诉你什么
函数签名的类型	返回什么类型（`M` = 同类型的 map；`bool` = 真假判断；`error` = 可能失败）
命名返回值	返回值的**角色**：`(updated bool, err error)` 这类命名本身就是文档，`updated` 的含义一眼可见
文档注释	返回值的**语义细节**——像 `Clone` 的“浅拷贝”、`Equal` 的“用 `==` 比较”都只写在这里
可运行示例	`go doc -ex <符号>`，示例比描述更不会歧义

再看一个返回 `bool` 的例子——注意注释里的限定条件：

```
$ godocbat maps.Equal
func Equal[M1, M2 ~map[K]V, K, V comparable](m1 M1, m2 M2) bool
    Equal reports whether two maps contain the same key/value pairs. Values are
    compared using ==.
```

Listing 10: 返回 bool，但“值用 == 比较”这条限定意味着：值类型必须可比较，含切片/函数的值类型用不了它

⚠️ **陷阱预警** 只看签名不看注释，最常见的翻车方式是**漏掉限定条件**：`maps.Equal` 要求 `V comparable`（值必须可比较），所以 `map[string][]byte` 用不了它——`[]byte` 不可比较。这类约束在签名的类型约束里写着（`V comparable`），但为什么不能用的解释往往在注释里。

3.3. 陷阱与易错

1. **把 runtime 内部函数当 API 查**。它们未导出、参数类型是 `unsafe.Pointer` 之流，看懂了也用不了；
2. **以为语言语法有“对应函数”**。`m[k]`、`delete` 没有函数形式，需要扩展功能时去查 `maps` / `slices`；
3. **只看签名不看注释**。返回值“是什么类型”在签名里，但“意味着什么”（浅拷贝？用 `==` 比较？会不会返回 nil?）在注释里；
4. **忽略类型约束那一长串**。`[M1, M2 ~map[K]V, K, V comparable]` 里的 `comparable` 不是装饰——它决定了哪些类型能用这个函数。

3.4. 关键点

📌 **核心要点** 两个问题要分开问：**“这段语法背后是什么机制”** → 查 runtime，用 `-gcflags=-S` + `go doc -u`；**“我该调用什么、返回值怎么用”** → 查配套标准库包，用 `go doc <包>.<函数>` 读签名 + 注释。

3.5. 变式检验

1. **换成切片**：同样的问题在 `slices` 包里有一整套答案（40 个导出符号），例如“删除元素”是 `slices.Delete`、“判断包含”是 `slices.Contains`；

2. **换成为什么 `map[string][]byte` 用不了 `maps.Equal`**：因为 `[]byte` 不可比较（切片不能做 `==`），所以不满足 `V comparable`——回到第一节讲的“map 键必须可比较”，是同一条规则的两种表现；
3. **换成想看示例代码**：`go doc -ex slices.Sort` 这类会带可运行示例（标准库的 `ExampleXxx` 函数），比读描述直观。

3.6. 同类归类

1. 与 Python 的对照：Python 的 `dict` 有内建方法（`.keys()`、`.update()`），而 Go 把这类功能拆到了 `maps` 包——这是 Go“语言核心保持小、功能放标准库”的一贯取舍；
2. 与 Rust 的对照：Rust 的 `Vec` / `HashMap` 方法直接挂在类型上（`vec.sort()`），Go 选择了“泛型函数 + 显式包”（`slices.Sort(vec)`）；两种风格各有取舍，Go 的好处是内置类型不必自带几十个方法；
3. 往下延伸：本课程后面会用 `sort`（对 SSTable 的键排序）、`encoding/binary`（编解码）、`bytes`（比较），都可以先用 `godocbat <包>` 扫一遍包概览建立印象，再 `godocsrc` 深入实现。

3.7. Go 语言机制补充

3.7.1. 为什么这些函数的签名那么长

`func Clone[M ~map[K]V, K comparable, V any](m M) M` 里的方括号是**类型参数**（泛型），读法可以拆成三块：

```
func Clone[M ~map[K]V, K comparable, V any](m M) M
           └──────────────────────────┘  └┘ └──┘
             M 是类型参数，约束为          参数  返回值
             "底层类型是 map[K]V 的任意类型"
```

Listing 11: 泛型签名的读法：方括号是“可以接受哪些类型”，圆括号才是“要传什么、返回什么”

`~` 这个符号表示“底层类型是它”（而不是“必须是它本身”），所以自定义的 `type MyMap map[string]int` 也能传进去。不写 `~` 就只能传精确类型。

3.7.2. 判断一个标识符“能不能用”的两条规则

情况	结果
小写开头（`makemap`、`mapassign_faststr`）	未导出，包外不可见——报 `undefined`
在 `internal/` 目录下的包	只有父目录树内能导入——报 `use of internal package ... not allowed`
大写开头且不在 `internal/`	可以用 ✓

这两条规则前面都单独讲过（导出规则、`internal` 规则），这里正好用 `runtime.makemap` 这个实例把它们串起来了。

📌 **核心要点** 一句话收束：**查 API 先问“这是谁的东西”**。 语言语法的问题找配套包（`maps` / `slices` / `strings` / `cmp`），库函数的问题看签名 + 注释两个信息源，而 runtime 里的东西只用于理解机制，不用于调用。

4. 怎么用 go doc 查 map 相关的内容

4.1. 费曼直觉

“查 map”这句话其实指向**四个不同的地方**，用错入口就会查不到东西：

想查什么	命令	说明
内建操作（`make` / `delete` / `clear` / `len`）	`go doc builtin`	预声明标识符的总入口；再用 `go doc builtin.delete` 看单个
map 的实用函数（`Clone` / `Equal` / `Keys`）	`go doc maps`	配套包，10 个函数
并发安全的 map	`go doc sync.Map`	附带“什么时候该用它”的说明
map 的**语言语义**（键必须可比较等）	**go doc 查不到**	在语言规范里：`/usr/lib/go/doc/go_spec.html` 的 Map types 一节

4.2. 硬核机制

4.2.1. 三个反直觉的实测结果

第一次查 map 的人几乎都会踩这几下：

```
# ① 直接查内建类型——失败
$ go doc map
doc: no Go files in /home/zz/project/db-go/note

# ② 当作 builtin 里的符号——也失败
$ go doc builtin.map
doc: no symbol map in package builtin

# ③ 但内建函数能查
$ go doc builtin.delete
func delete(m map[Type]Type1, key Type)
    The delete built-in function deletes the element with the specified key
    (m[key]) from the map. If m is nil or there is no such element, delete is a
    no-op.
```

Listing 12: go doc map 失败，go doc builtin.delete 成功——差别在哪，见下

原因有两层：

1. `go doc map` 失败是因为**参数被当成包路径**去解析了——仓库里没有叫 `map` 的目录，于是报 `no Go files`。它不会去理解“你想查的是内建类型”；
2. `go doc builtin.map` 失败是因为 `map` **本身在** `builtin` **里没有条目**：它是**类型构造器**（`map[K]V` 需两个参数才成类型），而 `builtin` 只收录可独立命名的预声明标识符。所以 `make`、`delete`、`clear` 有条目，`map` 没有。

📌 **核心要点** 查内建东西的正确入口是 `go doc builtin`（不带参数），先看清单再查单个。清单里与 map 有关的函数是：`make`（创建）、`delete`（删键）、`clear`（清空）、`len`（长度），以及挂在通用类型下的 `cap`。

4.2.2. 路径一：内建操作 —— go doc builtin

```
$ go doc builtin | head -16
package builtin // import "builtin"

Package builtin provides documentation for Go's predeclared identifiers.
The items documented here are not actually in package builtin but their
descriptions here allow godoc to present documentation for the language's
special identifiers.

const true = 0 == 0 ...
const iota = 0
func clear[T ~[]Type | ~map[Type]Type1](t T)
func close(c chan<- Type)
func delete(m map[Type]Type1, key Type)
func max[T cmp.Ordered](x T, y ...T) T
func min[T cmp.Ordered](x T, y ...T) T
func panic(v any)
func print(args ...Type)
```

Listing 13: builtin 包的第一段说明很关键：这些标识符“其实不在 builtin 包里”，只是借这个包给它们安个家

其中两个最值得记的：

```
$ go doc builtin.clear
func clear[T ~[]Type | ~map[Type]Type1](t T)
    The clear built-in function clears maps and slices. For maps, clear deletes
    all entries, resulting in an empty map. ... If t is nil, clear is a no-op.

$ go doc builtin.make | grep -A 2 'Map:'
      - Map: An empty map is allocated with enough space to hold the specified
        number of elements. The size may be omitted, in which case a small
        starting size is allocated.
```

Listing 14: clear（Go 1.21+）能一次清空 map，比逐个 delete 简洁；make 的容量提示也在文档里

4.2.3. 路径二：实用函数 —— go doc maps

内置语法只提供三种操作（`m[k]`、`m[k]=v`、`delete`），其余需求全在 `maps` 包里：

```
$ godocbat maps
package maps // import "maps"

Package maps defines various functions useful with maps of any type.

func Clone[M ~map[K]V, K comparable, V any](m M) M
func Collect[K comparable, V any](seq iter.Seq2[K, V]) map[K]V
func Copy[M1 ~map[K]V, M2 ~map[K]V, K comparable, V any](dst M1, src M2)
func DeleteFunc[M ~map[K]V, K comparable, V any](m M, del func(K, V) bool)
func Equal[M1, M2 ~map[K]V, K, V comparable](m1 M1, m2 M2) bool
func Keys[Map ~map[K]V, K comparable, V any](m Map) iter.Seq[K]
func Values[Map ~map[K]V, K comparable, V any](m Map) iter.Seq[V]
... (共 10 个)
```

Listing 15: godocbat 就是前面配的那个 fish 函数，等价于 go doc + bat 高亮

4.2.4. 路径三：并发安全 map —— go doc sync.Map

```
$ godocbat sync.Map
type Map struct {
  // Has unexported fields.
}
    Map is like a Go map[any]any but is safe for concurrent use by multiple
    goroutines without additional locking or coordination. Loads, stores,
    and deletes run in amortized constant time.

    The Map type is specialized. Most code should use a plain Go map instead,
    with separate locking or coordination, for better type safety and to make it
    easier to maintain other invariants along with the map content.

    The Map type is optimized for two common use cases: (1) when the entry for
    a given key is only ever written once but read many times, as in caches
    that only grow, or (2) when multiple goroutines read, write, and overwrite
    entries for disjoint sets of keys.
```

Listing 16: 文档自己就说了“大多数代码应该用普通 map + 自己的锁”——这句提示比签名更有价值

> ⚠ **陷阱预警** 别看到 `sync.Map` 就当“map 的加强版”用。它的文档明确写着 `The Map type is specialized. Most code should use a plain Go map instead` ——只在“读多写一次”（缓存）或“多个 goroutine 操作互不相交的键集”这两类场景下才值得用。其余情况用普通 map 加 `sync.Mutex` / `sync.RWMutex` 更清晰、类型也更安全。

4.2.5. 路径四：语言语义 —— 只能看规范

`go doc` 管的是**标准库 API**；而“map 类型本身是什么、有什么限制”属于**语言语义**。最典型的一条——键必须可比较——在规范里是这样写的（本机 `/usr/lib/go/doc/go_spec.html`，Map types 一节）：

```
A map is an unordered group of elements of one type, called the element type,
indexed by a set of unique keys of another type, called the key type.
The value of an uninitialized map is nil.
```

```
MapType = "map" "[" KeyType "]" ElementType .

The comparison operators == and ≠ must be fully defined for operands of the
key type; thus the key type must not be a function, map, or slice.
```

最后那句正是“`[]byte` 不能做 map 键”的依据。`go doc` 里查不到这类规则——**它是语言的公理，不是某个包的 API。**

4.3. 陷阱与易错

1. `go doc map`。参数会被当包路径；查内建标识符要走 `go doc builtin`；
2. **以为 `builtin` 里有 `map` 条目。**它是类型构造器，不是可命名的具名类型；
3. **只查标准库，忘了规范。**“键必须可比较”这类规则永远不在 `go doc` 里；
4. **把 `sync.Map` 当默认选择。**文档自己劝退：大多数场景用普通 map + 锁。

4.4. 关键点

> 📌 **核心要点** 查 map 的四条路径：**内建操作** → `go doc builtin`；**实用函数** → `go doc maps`；**并发场景** → `go doc sync.Map`；**语言语义** → 语言规范（`/usr/lib/go/doc/go_spec.html`）。记住分界线：`go doc` 答“库有什么”，规范答“语言允许什么”。

4.5. 变式检验

1. **换成切片**：同样四条路径成立——`go doc builtin`（`append` / `copy` / `len` / `cap` / `clear`）、`go doc slices`（40 个函数）、并发场景不需要（切片本身不涉及并发语义）、规范里 Slice types 一节；
2. **换成想知道 map 在内存里长什么样**：`go doc internal/runtime/maps` 能看到 Swiss table 的文档，但记住那是**内部实现**，只能读不能调；
3. **换成想知道某函数能不能用在你的类型上**：查它的类型约束（`K comparable`），再对照规范里该类型是否可比较。

4.6. 同类归类

1. 与查标准库普通包的区别：普通包像 `strings`、`fmt`，直接 `go doc strings` 即可；而 map / slice / chan 这类**语言内置结构**要分三层查（builtin → 配套包 → 规范）；
2. 与 Python 的对照：Python 的 `dict` 用 `help(dict)` 一次就能看到所有方法（因为方法挂在类型上）；Go 因为把功能拆到了 `maps` 包，所以要多查一处——这是“语言小、库大”的代价与收益；
3. 往下延伸：本课程后面会用 `map[string][]byte` 做内存 KV（就是你现在的 `KV.mem`），再往后会换成磁盘存储——那时 `maps` 包里的 `Clone` / `Copy` 在“把内存状态刷到磁盘”的路径上会有用武之地。

4.7. Go 语言机制补充

4.7.1. builtin 包是个“文档用的假包”

`go doc builtin` 的说明写得很清楚：

```
Package builtin provides documentation for Go's predeclared identifiers.
The items documented here are not actually in package builtin but their
descriptions here allow godoc to present documentation for the language's
special identifiers.
```

也就是说：你**不能** `import "builtin"`（虽然没有编译器阻止，但里面没有可用的代码）；它存在的唯一目的是让 `len` 、 `make` 、 `delete` 这些**语言关键字**有一份能查询的文档。这是一处很典型的“文档基础设施”设计——为了可查询而造一个假包。

4.7.2. 预声明标识符与关键字的区别

顺带厘清一组容易混淆的概念：

类别	例子	能否在 BUILTIN 查到
关键字（keyword）	`for` 、 `if` 、 `func` 、 `range` 、 `type` 、 `go`	不能——它们不是标识符
预声明标识符	`len` 、 `make` 、 `delete` 、 `clear` 、 `append` 、 `true` 、 `nil`	能—— `go doc builtin` 收录
内建类型名	`int` 、 `string` 、 `error` 、 `any`	能——以 `type X ...` 形式列出
类型构造器	`map[K]V` 、 `[]T` 、 `chan T`	不能——需要参数才成类型

> 📌 **核心要点** 一句话收束： **`go doc` 查不到 `map` 不是 bug，是设计。** map 由三层构成——语言语义（规范）、内建操作（builtin）、实用函数（ `maps` 包）；按这三层去找，每一层都有明确的入口。

5. 逐段读懂一个泛型函数声明

5.1. 费曼直觉

Go 的函数声明可以拆成**四个固定位置**，从左到右依次是：

```
func    函数名    [类型参数]    (参数列表)    返回值
 ↑        ↑          ↑            ↑           ↑
关键字   名字      泛型声明      收什么      给什么
```

前三段里面只有“类型参数”是 Go 1.18 才有的新东西。我们以 `maps.All` 为例逐段拆：

```
1    func All[Map ~map[K]V, K comparable, V any](m Map) iter.Seq2[K, V]
```

Listing 17: 一行里有五个信息块，逐个看就不难了

5.2. 硬核机制

5.2.1. 第一段：func All

1. `func` 是关键字，声明“这是一个函数”，没有返回值前缀（不像 C 的 `int f()` ）；
2. `All` 是函数名。**首字母大写 = 导出**，所以能从别的包用 `maps.All` 调用；小写就只能在 `maps` 包内部用（前面讲过导出规则）。

5.2.2. 第二段：[Map map[K]V, K comparable, V any]

方括号里是**类型参数列表**（泛型）。规范里的原话把它的来历说得很直白：

```
The type parameter list looks like an ordinary function parameter list
except that the type parameter names must all be present and the list is enclosed
in square brackets rather than parentheses
```

翻译：它**长得像普通参数列表**，区别是**用方括号**、而且是“类型”层面的参数。这里声明了三个：

类型参数	约束（能接受什么类型）	为什么这么约束
`Map`	`~map[K]V`：底层类型是 `map[K]V` 的任意类型	这样才能用 `m` 做 map 操作（取键、遍历）
`K`	`comparable`：可比较的类型	map 的**键**必须可比较——这是语言规定，不是这个包的偏好
`V`	`any`：任何类型	值可以是任何东西，函数不假设它的性质

三个词要单独解释：

1. **`any`** —— 预声明标识符，规范里就是 `type any = interface{}` 的别名（ `go doc builtin.any` 能查到）。 `any` 和 `interface{}` 完全等价，前者只是更好读；
2. **`comparable`** —— 也是预声明的约束，`go doc builtin.comparable` 的说明是“由所有可比较类型实现的接口（布尔、数字、字符串、指针、channel、可比较类型的数组、字段全可比较的结构体）”。它**只能**用作类型参数的约束，不能当变量类型；
3. **`~`（波浪号）** —— 表示“**底层类型**是它”，而不是“必须是它本身”。看实测：

```go
type MyMap map[string]int // 自定义类型，底层类型是 map[string]int

func main() {
  m := MyMap{"a": 1, "b": 2}
  for k, v := range maps.All(m) { // 能传进去，因为约束写的是 ~map[K]V
    fmt.Println(k, v)
  }
}
```

Listing 18: 带 ~ 的约束允许自定义类型；若写成不带 ~ 的 map[K]V，MyMap 就会被拒绝

5.2.3. 第三段：(m Map)

普通参数列表。一个参数：名字 `m`，类型是上面声明的类型参数 `Map`。**注意 `Map` 和 `map` 只差大小写但完全不同**——`Map` 是我们自己起的类型参数名，`map` 是语言关键字。

5.2.4. 第四段：iter.Seq2[K, V]

这是“返回值”，也是整行里最需要展开的部分。查它的定义：

```
$ go doc iter.Seq2
type Seq2[K, V any] func(yield func(K, V) bool)
    Seq2 is an iterator over sequences of pairs of values, most commonly
    key-value pairs. When called as seq(yield), seq calls yield(k, v) for
    each pair (k, v) in the sequence, stopping early if yield returns false.
```

Listing 19: Seq2 本身就是一个函数类型——所以 maps.All 返回的不是“数据”，而是“一个会产生数据的函数”

把 `iter.Seq2[K, V]` 代入进去，`maps.All` 的返回值就等价于：

```
func(yield func(K, V) bool)      ← 一个接受回调、不返回值的函数
```

所以 `maps.All(m)` 得到的东西**可以像函数一样调用**：你把一个 `yield` 回调交给它，它就为每一对键值调用一次你的回调。

5.2.5. 怎么用这个返回值（两种写法，都实测过）

```go
m := map[string]int{"a": 1, "b": 2}

// 写法一：range-over-func (Go 1.23+)，日常首选
for k, v := range maps.All(m) {
  fmt.Println("range:", k, v)
}

// 写法二：手动调用，看清本质——它就是个接受回调的函数
maps.All(m)(func(k string, v int) bool {
  fmt.Println("yield:", k, v)
  return k != "b" // 返回 false 立刻停止遍历
})
```

Listing 20: 两种写法等价；yield 返回 false 是“提前停止”的机制

```
$ go run .
range: a 1
range: b 2
yield: a 1
yield: b 2
```

Listing 21: 实测输出：两种写法都完整遍历了一遍

> 📌 **核心要点** 读懂 `iter.Seq2[K, V]` 的关键：**它是一个函数类型，不是容器。** `maps.All(m)` 没有立刻遍历任何东西，它只是返回一个“待启动的遍历过程”——真正开始工作是在 `range` 或手动调用那一刻。

5.2.6. 从简到繁：函数声明的六个台阶

把上面这些拼起来，Go 函数声明的复杂度是这样长出来的：

```
func f()                                    // 1. 不收不给
func f(x int)                               // 2. 收一个
func f(x int) int                           // 3. 给一个
func f(x int) (int, error)                  // 4. 给两个（错误处理惯例）
func f[T any](x T) T                        // 5. 泛型：类型也当参数
func f[K comparable, V any](m map[K]V) []V  // 6. 多个类型参数 + 约束
```

Listing 22: 六行覆盖了绝大多数你会遇到的签名形状；`maps.All` 属于第 6 档再加一个函数类型返回值

5.3. 陷阱与易错

1. **把 `Map` 当关键字。**它只是这个函数自选的**类型参数名**，换成 `M` 或 `MapT` 都不影响含义（`M ~map[K]V` 效果一样）；
2. **以为 `~` 可有可无。**带 `~` 允许自定义类型（`type MyMap map[string]int`），不带则只认精确类型——这是“我的类型能不能传进去”的决定性差异；
3. **把 `any` 当成特殊类型。**它只是 `interface{}` 的别名，`go doc builtin.any` 一句话就说清了；
4. **以为 `comparable` 是万能约束。**它要求类型能参与 `==` 比较——所以 `[]byte`、`map`、函数都不是 `comparable`；
5. **把 `iter.Seq2` 当切片用。**它不是容器、不能索引、不能求长度，只能被 `range` 或手动调用一次性地消费。

5.4. 关键点

> 📌 **核心要点** 读 Go 泛型签名就四步：① **找方括号（类型参数）② 每个类型参数看约束（大写字母开头的或者 `any` / `comparable` 这类预声明约束）③ 看圆括号（真正的参数）④ 看返回值，如果是函数类型（如 `iter.Seq2`）就展开它看回调的形状。**

5.5. 变式检验

1. **换成 `iter.Seq`**：单值版本，`type Seq[V any] func(yield func(V) bool)` —— `maps.Keys(m)` 返回的就是它，少了键、只遍历值；
2. **换成不带 `~` 的约束**：把 `Map ~map[K]V` 改成 `Map map[K]V`，前面那个 `MyMap` 例子就会编译失败（`MyMap does not satisfy map[K]V`）——这是观察 `~` 作用的最快实验；
3. **换成一个你自己写的泛型函数**：`func First[V any](s []V) (V, bool)` 是最小可用的练习——一个类型参数、一个约束、两个返回值。

5.6. 同类归类

1. 与 C++ 模板的对照：C++ 的 `template<typename T> T max(T a, T b)` 在**实例化时**才检查约束（报错常常又长又难读）；Go 把约束**写在签名里**（`T cmp.Ordered`），编译期即校验，报错位置就在声明处；
2. 与 Java 泛型的对照：Java 用 `extends` 表达上界（`<T extends Comparable<T>>`），Go 用接口当约束（`T comparable`）——两者都是“给类型参数划范围”，Go 的写法更接近普通接口；
3. 往下延伸：本课程后面会实现存储引擎的通用层，可能写出 `func Merge[Iter any](iters []Iter) Iter` 这类泛型函数；到那时你会发现约束怎么写，取决于你打算在函数体里对这些类型做什么操作。

5.7. Go 语言机制补充

5.7.1. 约束本质上是一个接口

规范里说得很清楚（本机 `/usr/lib/go/doc/go_spec.html`，Type constraints 一节）：

```
A type constraint is an interface that defines the
set of permissible type arguments for the respective type parameter and controls the
operations supported by values of that type parameter

[T []P]                    // = [T interface{[]P}]
[T ~int]                   // = [T interface{~int}]
[T int|string]             // = [T interface{int|string}]
```

也就是说：`K comparable` 完整写法是 `K interface{ comparable }`，`~map[K]V` 完整写法是 `~map[K]V` 包在 `interface{}` 里。日常写法只是省略了外层的 `interface{}`。

“约束控制可用的操作”这句话很关键：因为 `K` 被约束为 `comparable`，函数体里才能对 `K` 类型的值做 `==` 比较、才能把它当 map 的键；如果约束写成 `any`，那么连 `==` 都不允许。

5.7.2. 类型参数列表的语法

```
TypeParameters = "[" TypeParamList [ "," ] "]" .
TypeParamList  = TypeParamDecl { "," TypeParamDecl } .
TypeParamDecl  = IdentifierList TypeConstraint .
```

读法：方括号里是一串“名字 + 约束”，用逗号分隔。`IdentifierList` 意味着**多个类型参数可以共享同一个约束**，例如 `[K, V comparable]` 等价于 `[K comparable, V comparable]`。

> 📌 **核心要点** 一句话收束：`func 名字 [类型参数] (参数) 返回值` ——**方括号管“能接受哪些类型”，圆括号管“收什么值”，返回值里如果是函数类型就再展开一层。**

6. 类型参数（泛型）到底是做什么用的

6.1. 费曼直觉

先问一个具体问题：**写一个“求切片最大值”的函数，你怎么让它对 `[]int`、`[]float64`、`[]string` 都能用？**

这个问题在 Go 1.18 之前只有两种答法，都不理想；泛型是第三种答法。三条路摆在一起看，泛型的意义立刻清楚了。

6.2. 硬核机制

6.2.1. 三条路：写三遍、用 any 硬塞、用泛型

```
// A: 没有泛型——每种类型各写一遍（函数体完全重复）
func MaxInt(s []int) int {
  m := s[0]
  for _, v := range s[1:] {
    if v > m {
      m = v
    }
  }
  return m
}

func MaxFloat64(s []float64) float64 {
  m := s[0]
  for _, v := range s[1:] {   // 和上面一模一样，只是类型不同
    if v > m {
      m = v
    }
  }
  return m
}

// B: 用 any 硬塞——一份代码，但类型信息丢了
func MaxAny(s []any) any {
  m := s[0]
  for _, v := range s[1:] {
    if v.(int) > m.(int) { // 必须断言，而且写死了 int
      m = v
    }
  }
  return m
}

// C: 泛型——一份代码，类型保留
func Max[T cmp.Ordered](s []T) T {
  m := s[0]
  for _, v := range s[1:] {
    if v > m { // 直接比较，编译器知道 T 可排序
      m = v
    }
  }
  return m
}
```

Listing 23: 同一个算法的三种写法：A 靠复制、B 靠放弃类型、C 把类型也变成参数

6.2.2. 实测：B 和 C 的差别在哪

先看用法——A 要记住每个类型对应的函数名，C 一份通吃：

```
    fmt.Println("A 两个版本:", MaxInt([]int{3, 1, 2}), MaxFloat64([]float64{3.5, 1.2}))
    fmt.Println("C 一份代码:", Max([]int{3, 1, 2}), Max([]float64{3.5, 1.2}),
Max([]string{"b", "a", "c"}))

    // $ go run .
    // A 两个版本: 3 3.5
    // C 一份代码: 3 3.5 c
```

Listing 24: C 版本对三种类型都能用，且返回值类型就是原类型（不是 any）

再看传错类型时的表现——这是最能说明“类型安全”的地方：

```
    # B（any 版）：把 float64 传进去，编译通过，运行时崩
    $ go run .
    B 传 int 可以: 3
    B 传 float64 会怎样:
    panic: interface conversion: interface {} is float64, not int

    # C（泛型版）：传不可排序的类型，编译期就拦下，还告诉你合法集合
    $ go build ./...
    ./check.go:4:12: bool does not satisfy cmp.Ordered (bool missing in ~int | ~int8 | ~int16
| ~int32 | ~int64 | ~uint | ... | ~string)
```

Listing 25: 一个在深夜炸在线上，一个在敲代码时就被拦下——这是泛型最实际的价值

最后看性能——同样遍历 1000 个 `int` 求最大值（`[]any` 预先构造好，不计入耗时）：

```
    $ go test -bench=. -benchmem
    BenchmarkMaxGeneric-12    2699258    438.3 ns/op    0 B/op    0 allocs/op
    BenchmarkMaxAny-12        1000000   1112 ns/op      0 B/op    0 allocs/op
```

Listing 26: 泛型版快约 2.5 倍：any 版每次比较都要做一次接口断言，泛型版直接比较

6.2.3. 于是“类型参数”是什么，可以一句话回答

它把“类型”也变成函数的一个参数。普通参数让你换**值**，类型参数让你换**类型**——同一份逻辑，服务多种类型。

写法	解决了什么	代价
`func MaxInt(s []int)`	针对一种类型，最快最直白	换类型就得复制一份函数体
`func MaxAny(s []any)`	一份代码通吃	类型信息丢失：用时要断言、错误推迟到运行时、断言有开销
`func Max[T cmp.Ordered](s []T)`	一份代码 + 类型保留 + 编译期检查	签名变长：约束需要设计

📌 **核心要点** 泛型不是“少写几行”的语法糖，它同时买到三样东西：① **代码只写一份** ② **类型在编译期就是确定的（返回值还是 `T`，不是 `any`）** ③ **没有接口装箱与断言的开销。**第 ② 条最关键——它把错误从“运行时 panic”提前到了“编译期报错”。

6.2.4. 但别急着到处用：标准库自己就很克制

数一下官方自己用泛型的规模：

```
maps            10 个泛型函数
slices          40 个泛型函数
cmp             3 个泛型函数
iter            2 个泛型函数

fmt             0 个泛型函数
os              0 个泛型函数
net/http        0 个泛型函数
encoding/json   0 个泛型函数
```

Listing 27: 泛型集中在“容器与算法”包；业务型包一个泛型函数都没有

这不是巧合，而是 Go 社区的明确建议：**能用具体类型就用具体类型；能用接口就用接口；只有确实需要“同一逻辑服务多种类型”时才上泛型。**

6.2.5. 接口与泛型的分工

两者常被混用，其实解决的是不同问题：

手段	回答的问题	典型例子
接口	**能做什么**——只要实现了某些方法就行，不关心具体类型	`io.Reader`：任何有 `Read` 方法的类型都行
泛型	**类型之间的关系**——输入输出同类型、需要比较或排序	`maps.Clone[M ~map[K]V](m M) M`：进出同一种 map

一个实用的判断法：**如果你在函数体内不需要知道“具体是哪个类型”，只需要它会某些方法——用接口；如果你需要“这个类型和那个类型是同一个”，或者要对值做比较、排序——用泛型。**

6.3. 陷阱与易错

1. **为了“通用”而通用。**只有一个调用点、只有一种类型的函数，加类型参数纯属给自己添麻烦；
2. **用 `any` 代替泛型。**像 `MaxAny` 那样把类型擦掉，等于把编译期检查换成了运行时 panic（上面的实测就是活例）；
3. **以为泛型一定更快。**快是因为**避免了装箱与断言**，不是泛型本身有什么魔法——如果泛型版内部仍要转成接口，优势就没了；
4. **约束写得太宽。**把 `T cmp.Ordered` 写成 `T any`，函数体里的 `v > m` 会直接编译失败——约束不只是“限制”，它**决定了函数体内允许哪些操作。**

6.4. 关键点

> 📌 **核心要点** 类型参数 = **把类型当参数。**它买到的是三段合一：一份代码、编译期类型安全、无装箱开销。判据是“同一种逻辑要不要服务多种类型”——是，才用泛型；不是，写具体类型或用接口。

6.5. 变式检验

1. **换成本课程的场景**：你现在 `KV` 的 `mem` 是写死的 `map[string][]byte`；如果将来想做“值可以是任意类型”的 KV，才会考虑把 `V` 参数化（但要先想清楚序列化问题——那才是难点）；
2. **换成排序算法**：`slices.Sort[T cmp.Ordered](s []T)` 是标准库的量产泛型函数，把 `cmp.Ordered` 换成自定义接口，就能支持任意复杂类型；
3. **换成对比 Java/C++**：Java 的泛型靠**类型擦除**（运行时信息没了，所以不能 `new T()`）；C++ 的模板靠**编译期展开**（能 `new T()`，但错误信息极长）。Go 介于两者之间：编译期生成按“形状”共享的代码，因而没有反射开销，也不允许 `new(T)` 之外的动态构造。

6.6. 同类归类

1. 与“代码生成”的对照：在泛型出现前，Go 社区用 `go generate` 生成 `MaxInt` / `MaxFloat64`（例如标准库的 `sort` 早期就靠生成）——泛型把这件事收进了语言；
2. 与“接口 + 反射”的对照：`encoding/json` 这类包靠反射处理任意类型，代价是运行时开销；泛型适合“编译期已知类型集合”的场景，反射适合“运行时才知道类型”的场景（如解析未知 JSON）；
3. 往下延伸：本课程后面要写的存储引擎，多数地方用具体类型更清楚（`[]byte` 键、`[]byte` 值）；真正可能用到泛型的是“多路归并迭代器”这类纯算法组件。

6.7. Go 语言机制补充

6.7.1. 泛型的实例化方式：按“形状”共享代码

编译器不会为每个具体类型都生成一份独立代码（那是 C++ 模板的做法，会让二进制膨胀），也不像 Java 那样完全擦除（那样每次操作都要装箱）。Go 的做法叫 **gcshape stenciling**：按类型参数在运行时的**形状**（例如“是指针”、“是 8 字节标量”）生成共享版本。

这解释了上表的实测结果：`Max[int]` 与 `Max[int64]` 很可能共用同一份机器码（形状相同），而 `any` 版每次比较都要走接口断言（查 itab、比较类型），于是慢了 2.5 倍。

6.7.2. 约束决定了函数体内能做什么

这一点比“限制调用者”更重要。规范里说约束“**controls the operations supported by values of that type parameter**”——因为 `T` 被约束为 `cmp.Ordered`，`v > m` 才合法；换成 `any`，同样的代码就会报 `invalid operation: v > m (type T is not comparable)`。

所以读泛型函数的顺序应该是反的：**先看函数体里对它做了什么操作，再回头看约束为什么这样写**——两者必须自洽。

> 📌 **核心要点** 一句话收束：**类型参数存在的唯一理由是“同一逻辑 × 多种类型”**。 它换来编译期类型安全与接近原生的性能；如果一个函数只服务一种类型，那它就不需要类型参数。

7. `kv.mem[string(key)]` 到底返回什么

7.1. 费曼直觉

你写的这一行，**编译器自己就把答案说出来了**：

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
  val, ok, err := kv.mem[string(key)]
  return val, ok, err
}
```

Listing 28: 当前的写法：把三个变量接到一个 map 索引上

```
$ go build ./...
internal/db/kv.go:15:15: no new variables on left side of :=
internal/db/kv.go:15:18: assignment mismatch: 3 variables but 1 value
```

Listing 29: 第二条错误直接回答了你的问题：它只产生 1 个值（在“三个变量”这种写法下）

一句话结论：**map 索引最多返回两个值——值本身和“是否存在”，永远不会有 error**。三个变量接不住它。

7.2. 硬核机制

7.2.1. map 索引的返回值规则（规范原文）

语言规范 Index expressions 一节写得非常明确：

```
For a of map type M:

  if the map contains an entry with key x,
      a[x] is the map element with key x
      and the type of a[x] is the element type of M
  if the map is nil or does not contain such an entry,
      a[x] is the zero value
      for the element type of M
```

注意它**只字未提 error**——键不存在时返回的是**零值**，不是错误。`map[string][]byte` 的零值是 `nil`，所以 `kv.mem["不存在的键"]` 会得到 `nil`。

双值形式是另一个“特例形式”，同一节的原文：

```
An index expression on a map a of type map[K]V
used in an assignment statement or initialization of the special form

v, ok = a[x]
v, ok := a[x]
var v, ok = a[x]

yields an additional untyped boolean value. The value of ok is
true if the key x is present in the map, and
false otherwise.
```

关键在 **the special form**：双值形式**只在那三种写法下有效**（恰好两个变量）。所以三个变量时，编译器按普通单值形式处理，于是报 `3 variables but 1 value`。

7.2.2. 于是有且只有两种写法

```
    m := map[string][]byte{"k": []byte("v")}

    // 写法一：单值——不关心键在不在，不在就拿到零值（nil）
    v := m["nope"]                  // v == nil，不报错

    // 写法二：双值（comma-ok）——多一个「存在性」布尔
    v, ok := m["nope"]              // v == nil, ok == false
```

Listing 30: map 索引就这两种形态；第三种写法不存在，因为 map 索引不会失败

📌 **核心要点 `err` 不是从 map 索引来的，是从 `Get` 的签名来的。**你的 `Get` 声明了 `(val []byte, ok bool, err error)` 三个返回值——这是你自己定的**接口契约**：将来把存储换成磁盘时，“读盘失败”才需要 `err`；内存版本里它恒为 `nil`。map 索引本身永远不会给你 `err`。

7.2.3. 顺带盘点：Go 里只有这三种表达式能“给两个值”

这不只发生在 map 上——语言里能产生“额外一个布尔”的表达式一共三种，且**形态完全一致**（单值 / comma-ok）：

```
    // ① map 索引
    v1 := m["k"]        // 单值：不存在时给零值
    v2, ok := m["k"]    // 双值：ok = 键是否存在

    // ② 类型断言
    s1 := x.(string)     // 单值：断言失败会 panic
    s2, ok2 := x.(int)   // 双值：ok2 = 断言是否成功

    // ③ channel 接收
    n1 := <-ch           // 单值：没数据就阻塞等待
    n2, ok3 := <-ch      // 双值：ok3 = channel 是否已关闭且为空
```

Listing 31: 三种“comma-ok”表达式；它们的第二值语义各不相同，但形式一样

实测输出：

```
    $ go run .
    map  单值: 0 | 双值: 0 false
    断言 单值: hello | 双值: 0 false
    chan 单值: 1 | 双值: 0 false
```

Listing 32: 三种表达式在“取不到”时的表现：map 给零值、断言给零值+false、channel 给零值+false

⚠️ **陷阱预警** 这三者的**单值形式风险不同**：map 索引取不到时安静地给零值（最安全）；类型断言失败会 **panic**；channel 接收在没数据时会**永久阻塞**（实测时我就撞上了 `fatal error: all goroutines are asleep - deadlock!`，因为空且未关闭的 channel 上接收会一直等）。所以凡是不确定能否成功的地方，都应该用双值形式。

7.2.4. 那“怎么知道任意表达式的返回值”？三条路

回到你的元问题——**怎么知道某个东西会返回几个值、什么类型**：

表达式的种类	返回值由什么决定	怎么查
函数 / 方法调用	它的**签名**	`go doc <包>.<符号>`；自己的函数就看声明
语言内置表达式（索引 / 断言 / channel 接收）	**语言规范**规定，不能自定义	试编译器（写错数量会告诉你 “N variables but M values”），或查规范的 Index expressions / Type assertions / Receive operator 三节
多重赋值里的函数调用	签名返回几个就能接几个（最后一个可以是 comma-ok 的 `, ok`）	同上

> 📌 **核心要点** 最快的办法其实是**让编译器回答**：故意写错变量个数，报错会精确告诉你 “N variables but M values”。这比翻文档快，而且报错永远与你当前的 Go 版本一致。

7.2.5. 正确写法

`Get` 的正确写法是把 map 索引的结果赋给**已声明的**命名返回值（注意用 `=` 而不是 `:=`）：

```
1   func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
2     val, ok = kv.mem[string(key)]  // 两个变量接两个值；err 未被赋值，保持零值 nil
3     return
4   }
```

Listing 33: 正确版本：`=` 而非 `:=`（val/ok 已在签名里声明），err 自动为 nil

这里有两个连环错误要同时修：

1. **`:=` 改成 `=`**——`val`、`ok`、`err` 已经在签名里声明过了，再用 `:=` 会报 `no new variables on left side of :=`；
2. **三个变量改成两个**——第三个 `err` 没有来源，因为它本来就不是 map 索引的产物。

📌 **核心要点** 一句话收束：**map 索引给 1 个值（或 comma-ok 形式给 2 个），其中没有 error。** `Get` 的第三个返回值是你自己定的契约，内存实现里让它保持 `nil` 即可——那是留给将来磁盘实现的位置。

7.3. 陷阱与易错

1. **把自定义方法的返回值套到语言内置表达式上。**`KV.Get` 有 `err` 是因为签名这么写的；`kv.mem[k]` 没 `err` 是因为规范这么定的；
2. **用 `:=` 给命名返回值赋值。**签名里已经声明过，要用 `=`；
3. **以为 map 取不到键是“错误”。**它是正常的业务情形，用 `ok` 表达，不要造一个 `err` 出来；
4. **单值形式用在不确定的地方。**map 索引单值形式安全，但类型断言会 panic、channel 会阻塞——这三者风险不同，别因为形式一样就以为行为一样。

7.4. 变式检验

1. **换成 `Set`：**`kv.mem[string(key)] = val` 是**赋值**给 map 元素，不产生任何返回值；往 nil map 赋值会 panic（规范原文：`Assigning to an element of a nil map causes a run-time panic`）；
2. **换成判断“键是否存在”：**如果只想知道存在性、不取值，写 `if _, ok := m[k]; ok { … }`——用下划线丢弃值，这也是 comma-ok 的常见用法；
3. **换成想知道“取到的值是不是 nil”：**注意 `[]byte` 的零值就是 `nil`，所以 `v == nil` 分不清“键不存在”和“存了一个 nil 值”——要区分必须用 `ok`。

7.5. 同类归类

1. 与 Python 的对照：Python 用 `m.get(k)`（取不到给 `None`）或 `m.get(k, default)`；Go 用 comma-ok 形式替代，好处是“取不到”和“取到零值”可以区分；
2. 与 C++ 的对照：`std::map::operator[]` 在键不存在时**会插入**一个默认值（副作用！）；`find()` 才返回迭代器。Go 的 map 索引没有这个副作用，只读不写；
3. 往下延伸：本课程后面做 SSTable 查询时会反复用到 comma-ok——“找到就返回，没找到就走下一层”，这个模式会贯穿整个存储引擎。

7.6. Go 语言机制补充

7.6.1. 为什么 map 索引不返回 error

因为**键不存在是 map 的正常语义**，不是异常。用 `err` 表达“键不存在”会让每次取值都要写错误处理，而且调用方还得分辨“键不存在”与“真出错”——那才是真正的负担。

反过来说，什么时候 `KV.Get` 的 `err` 才有意义？当底层换成磁盘时——读文件可能失败（IO 错误、文件损坏），那时 `err` 才承载“查询本身失败了”，而 `ok` 依然表达“键不存在”。**两个返回值各管一件事**，这就是签名里同时有 `ok` 和 `err` 的原因。

7.6.2. comma-ok 只能在“赋值/初始化”位置生效

规范那句 **used in an assignment statement or initialization** 很关键：双值形式不能在别的地方用。例如不能写 `return kv.mem[k], ok` 这种把戏；也不能把 comma-ok 塞进函数调用的实参里。

> 📌 **核心要点** 一句话收束：**想知道一个表达式返回什么，先问它属于哪一类**——内置表达式（索引 / 断言 / channel）查规范或让编译器告诉你；函数调用查签名。`kv.mem[string(key)]` 属于前者，答案是最多两个值、没有 error。

8. 怎么查 `:=` 与 `=` 的区别

8.1. 费曼直觉

先说结论：**`go doc` 查不到运算符**——它管的是“标准库里有什么”，而 `:=`、`=` 是**语言语法**，属于另一本书：**语言规范**。

```
$ go doc ':='
doc: no Go files in /home/zz/project/db-go/note

$ go doc '='
doc: no Go files in /home/zz/project/db-go/note
```

Listing 34: 和之前查 map 时一样：参数被当成包路径，去仓库里找同名目录，自然找不到

所以查法要换一套：**API 查 `go doc`，语法查规范**（本机 `/usr/lib/go/doc/go_spec.html`）。

8.2. 硬核机制

8.2.1. 规范地图：19 个大章

规范共 19 章，日常最常翻的其实就五章，先记住位置：

章节	管什么	典型问题
6. Variables	变量、零值、短变量声明的**位置**	为什么零值是 nil
10. Declarations and scope	声明与作用域	`:=` 为什么在这里声明
11. Expressions	表达式：索引、断言、调用、运算符	`m[k]` 返回几个值
12. Statements	语句：赋值、if、for、return	`=` 赋值语句的规则
13. Built-in functions	`make` / `delete` / `clear` / `len`	`clear` 能干什么

`:=` 与 `=` 的区别横跨第 10、12 两章：**短变量声明**在 Declarations and scope，**赋值语句**在 Statements。

8.2.2. 规范的原文（两节对照）

赋值（`=`）——Statements 章，Assignment statements 一节：

```
An assignment replaces the current value stored in a variable
with a new value specified by an expression.

Assignment = ExpressionList assign_op ExpressionList .
assign_op  = [ add_op | mul_op ] "=" .

x = 1
*p = f()
a[i] = 23
```

短变量声明（`:=`）——Declarations and scope 章：

```
A short variable declaration uses the syntax:

ShortVarDecl = IdentifierList ":=" ExpressionList .

It is shorthand for a regular variable declaration
with initializer expressions but no types:

"var" IdentifierList "=" ExpressionList .

i, j := 0, 10
f := func() int { return 7 }
ch := make(chan int)
```

> 📌 **核心要点** 这两段话就把区别说完了：`=` **是“替换已存在变量里的值”**；`:=` **是“声明一个新变量并给它初始值”**——规范的原话是 `It is shorthand for a regular variable declaration`，也就是说 `i, j := 0, 10` **完全等价于** `var i, j int = 0, 10`（省略类型，由右边推导）。

8.2.3. 一句话区分

写法	语义	前提	能否用于新名字
`x = v`	赋值：把 `x` 里的值换掉	`x` 必须**已经存在**	不能——名字不存在就没法赋值
`x := v`	声明 + 初始化：创建一个新变量 `x`	至少有一个左边的名字是**新的**	能——这正是它的用途

8.2.4. 三条硬规则（都实测过）

规则一：只能在函数体内用

```go
package main

var pkgLevel := 1   // 包级作用域
```

Listing 35: 包级用它，语法层面就通不过

```
$ go build ./...
./a.go:3:14: syntax error: unexpected :=, expected =
```

Listing 36: 报的是 syntax error——不是类型问题，而是这种语法在包级根本不允许

规则二：左边至少要有一个新变量

```go
a, b := 1, 2
fmt.Println(a, b)

a, b := 3, 4 // a、b 都是旧的，没有新的
```

Listing 37: 第二次 := 时两个名字都已存在

```
$ go build ./...
./main.go:10:7: no new variables on left side of :=
```

Listing 38: 这就是你上一轮在 Get 里遇到的报错——原因完全相同

规则三：允许“部分重声明”，但重声明的那个不算新变量

规范原文把这件事讲得很细：

```
Unlike regular variable declarations, a short variable declaration may redeclare
variables provided they were originally declared earlier in the same block
(or the parameter lists if the block is the function body) with the same type,
and at least one of the non-blank variables is new.
As a consequence, redeclaration can only appear in a multi-variable short declaration.
Redeclaration does not introduce a new variable; it just assigns a new value to the original.
The non-blank variable names on the left side of := must be unique.

field1, offset := nextField(str, 0)
field2, offset := nextField(str, offset)  // redeclares offset
x, y, x := 1, 2, 3                        // illegal: x repeated
```

最后那行例子很值得记：**同一个 `:=` 里名字不能重复**（`x, y, x := …` 非法）。

8.2.5. 回到你的 `Get`：为什么必须用 `=`

把规范那段话套到上一轮的代码上，一切就顺了：

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
  val, ok = kv.mem[string(key)]   // ☑ 正确：val/ok 已在签名（函数体的参数列表）里声明
  return
}
```

Listing 39: 签名里的命名返回值，规范上属于“函数的参数列表”，所以它们已经被声明过了

签名里的 `val`、`ok`、`err` 属于规范说的 **the parameter lists if the block is the function body**——**它们早就是已声明的变量**，所以：

1. 用 `:=` → 没有新变量（三个都是旧的）→ `no new variables on left side of :=`；
2. 用 `=` → 给已存在的变量赋值 → 正确。

> ⚠️ **陷阱预警** 一个容易混的变体：在函数体里写 `val, ok, err := kv.mem[string(key)]` 时，如果 `err` 是**新**名字（比如该函数签名里没声明 err），那么这一行**部分合法**——但 map 索引依然只给两个值，所以还是会报 `assignment mismatch: 3 variables but 2 values`。**“至少一个新变量”和“右边给几个值”是两条独立的检查**，别把它们混成一条。

8.3. 陷阱与易错

1. **拿 `go doc` 查语法**。它只认包路径，运算符、关键字、语法结构一律查不到；
2. **以为 `:=` 只是“更短的 `=`”**。它们语义不同：一个是声明，一个是赋值——`:=` 会引入新名字，`=` 不会；

3. **在 `if` / `for` 的初始化语句里误用 `=`**。那些位置正需要 `:=` 来声明局部变量（`if v, ok := m[k]; ok { … }`），用 `=` 会因为变量未声明而报错；
4. **以为 `:=` 的重复名字都会被重新声明**。规范说得很清楚：重声明的那个**不引入新变量**，只是给它赋值。

8.4. 关键点

> 📌 **核心要点** 两条记忆锚点：① **`go doc` 查“标准库有什么”，规范查“语言允许什么”**；② **`:=` 是 `var` 的简写（声明），`=` 是赋值（改值）**。判断用哪个，只看一件事：**左边的名字是不是已经存在了**——存在用 `=`，不存在用 `:=`（且至少要有一个不存在）。

8.5. 变式检验

1. **换成 `var` 显式声明**：`var val, ok = kv.mem[string(key)]` 与 `val, ok := kv.mem[string(key)]` 效果相同——前者是后者被“展开”的样子（这也是规范那句 shorthand 的字面意思）；
2. **换成 `range`**：`for k, v := range m` 里的 `:=` 同样是短变量声明（每次迭代都是新变量，这也解释了 Go 1.22 之前著名的“循环变量捕获”问题）；
3. **换成类型断言里的 `:=`**：`if v, ok := x.(T); ok { … }` 是标准写法，`ok` 的作用域仅限于该 `if`——这正是“声明”带来的作用域规则，用 `=` 就达不到。

8.6. 同类归类

1. 与 Python 的对照：Python 只有 `=`，且它在首次出现时就是“声明 + 赋值”（没有独立的声明语法）——所以 Python 里没有 `:=` 的对应物（`:=` 海象运算符是另一样东西，用于表达式内赋值）；
2. 与 C 的对照：C 的 `int x = 1;` 是“声明 + 初始化”，`x = 1;` 是赋值——这正是 Go 用 `:=` 与 `=` 分开表达的两件事；
3. 往下延伸：本课程后面写存储引擎时，`if v, ok := ...; ok` 与 `for k, v := range ...` 这两个模式会反复出现；把它们的差异记牢（新变量 vs 已存在变量），能省下大量“为什么报 no new variables”的时间。

8.7. Go 语言机制补充

8.7.1. 怎么在规范里快速定位

规范是一个 296KB 的 HTML 文件，直接翻很痛苦。三个实用技巧：

```
    # ① 先看大章地图（19 章）——知道该往哪一章找
    $ python3 -c "import re,html; s=open('/usr/lib/go/doc/go_spec.html').read();
print([html.unescape(re.sub(r'<[^>]+>','',m)) for m in re.findall(r'<h2[^>]*>(.*?)</h2>', s)])"

    # ② 用关键符号去搜（去掉标签后搜更可靠）
    $ grep -o 'ShortVarDecl[^<]*' /usr/lib/go/doc/go_spec.html | head -3

    # ③ 用语法产生式的名字搜——规范里每个语法都有个正式名字
    #    ShortVarDecl / Assignment / IndexExpr / TypeAssertion / ReceiveExpr
```

Listing 40: 三条定位技巧；第 ③ 条最有用：知道产生式名字就等于知道搜索引擎关键词

8.7.2. 规范里语法产生式的命名规律

规范的每个语法结构都有正式名字，命名规律很一致：

产生式名	对应什么
`ShortVarDecl`	短变量声明 `:=`
`Assignment`	赋值语句 `=`
`IndexExpr`	索引表达式 `a[x]`
`TypeAssertion`	类型断言 `x.(T)`
`ReceiveExpr`	channel 接收 `<-ch`
`CompositeLit`	复合字面量 `T{...}`

> 📌 **核心要点** 一句话收束：**`go doc` 回答“库有什么 API”，语言规范回答“语言允许什么”**。 遇到运算符、关键字、语法结构的问题（ `:=` / `=` / `range` / `select` ），直接去 `/usr/lib/go/doc/go_spec.html` ，用“章节标题”或“语法产生式名”搜，比搜索引擎更准、也永远不会版本错位。

Index of Listings

Listing 1 当前唯一的编译错误：Get 的函数体为空 5
Listing 2 命名返回值 + 裸 return：val/ok 已被赋值，err 自动为零值 nil 5
Listing 3 六种 map 操作的探针程序 7
Listing 4 六种语法 → runtime 调用，一一对应 8
Listing 5 整个字面量都在操作 SP（栈指针）——这是栈上分配的 map，没有堆分配、没有 makemap 调用 9
Listing 6 试着直接调用上一节看到的 runtime 函数 12
Listing 7 报的是 undefined——原因正是前面讲过的导出规则：小写开头，外部看不见 12
Listing 8 从“有什么”到“怎么用”的三步 13
Listing 9 签名说“返回 M”，注释说“是浅拷贝”——后者才是决定你怎么用的关键 13
Listing 10 返回 bool，但“值用 == 比较”这条限定意味着：值类型必须可比较，含切片/函数的值类型用不了它 14
Listing 11 泛型签名的读法：方括号是“可以接受哪些类型”，圆括号才是“要传什么、返回什么” 15
Listing 12 go doc map 失败，go doc builtin.delete 成功——差别在哪，见下 17
Listing 13 builtin 包的第一段说明很关键：这些标识符“其实不在 builtin 包里”，只是借这个包给它们安个家 18
Listing 14 clear（Go 1.21+）能一次清空 map，比逐个 delete 简洁；make 的容量提示也在文档里 18
Listing 15 godocbat 就是前面配的那个 fish 函数，等价于 go doc + bat 高亮 19
Listing 16 文档自己就说了“大多数代码应该用普通 map + 自己的锁”——这句提示比签名更有价值 19
Listing 17 一行里有五个信息块，逐个看就不难了 22
Listing 18 带 [K comparable, V any] 的约束允许自定义类型；若写成不带 [K comparable, V any] 的 map[K]V，MyMap 就会被拒绝 23
Listing 19 Seq2 本身就是一个函数类型——所以 maps.All 返回的不是“数据”，而是“一个会产生数据的函数” 23
Listing 20 两种写法等价；yield 返回 false 是“提前停止”的机制 23
Listing 21 实测输出：两种写法都完整遍历了一遍 24
Listing 22 六行覆盖了绝大多数你会遇到的签名形状；`maps.All` 属于第 6 档再加一个函数类型返回值 24
Listing 23 同一个算法的三种写法：A 靠复制、B 靠放弃类型、C 把类型也变成参数 28
Listing 24 C 版本对三种类型都能用，且返回值类型就是原类型（不是 any） 29
Listing 25 一个在深夜炸在线上，一个在敲代码时就被拦下——这是泛型最实际的价值 29
Listing 26 泛型版快约 2.5 倍：any 版每次比较都要做一次接口断言，泛型版直接比较 29
Listing 27 泛型集中在“容器与算法”包；业务型包一个泛型函数都没有 30
Listing 28 当前的写法：把三个变量接到一个 map 索引上 32

Listing 29 第二条错误直接回答了你的问题：它只产生 1 个值（在“三个变量”这种写法下） 32
Listing 30 map 索引就这两种形态；第三种写法不存在，因为 map 索引不会失败 33
Listing 31 三种“comma-ok”表达式；它们的第二值语义各不相同，但形式一样 33
Listing 32 三种表达式在“取不到”时的表现：map 给零值、断言给零值+false、channel 给零值+false 33
Listing 33 正确版本：`=` 而非 `:=`（val/ok 已在签名里声明），err 自动为 nil 34
Listing 34 和之前查 map 时一样：参数被当成包路径，去仓库里找同名目录，自然找不到 37
Listing 35 包级用它，语法层面就通不过 38
Listing 36 报的是 syntax error——不是类型问题，而是这种语法在包级根本不允许 38
Listing 37 第二次 := 时两个名字都已存在 38
Listing 38 这就是你上一轮在 Get 里遇到的报错——原因完全相同 39
Listing 39 签名里的命名返回值，规范上属于“函数的参数列表”，所以它们已经被声明过了 39
Listing 40 三条定位技巧；第 ③ 条最有用：知道产生式名字就等于知道搜索引擎关键词 40